TAREA #1 - ESTRUCTURAS DE DATOS

Lista Doblemente Enlazada de Productos

Sistema de Inventario, Reabastecimiento y Análisis de Importación para un Supermercado (Implementación en Python puro)

Luis Diego Fernández Quesada

Juan Alexander Aguilar Artola

Requerimientos del Proyecto



Lista Doble

Estructura principal para almacenar los productos con punteros doblemente enlazados (anterior y siguiente).



Cola FIFO Propia

Conversión automática a cola de suministros para productos agotados (existencias = 0) sin librerías externas.



Análisis & Reporte

Estadísticas de frecuencia por país de origen y exportación de reporte de recuperación financiera en .txt.

Estructura y Arquitectura

Lista Doblemente Enlazada

Cada nodo almacena los atributos clave del producto y mantiene doble referencia para recorrido bidireccional:

- **Datos:** ID, Nombre, Precio, País, Existencias.
- **Puntero Anterior:** Referencia al nodo previo.
- **Puntero Siguiente:** Referencia al nodo posterior.
- **Zero-External-Libs:** Construido desde cero con clases nativas.



https://techvidvan.com/tutorials/wp-content/uploads/2021/06/Doubly-linked-list-in-DS-1.jpg

Source: techvidvan.com

1. Nodo de la Lista: Clase Producto

Explicación del Nodo

Representa cada producto registrado en el supermercado. Inicializa sus referencias anterior y siguiente en None.

- Conversión explícita de tipos (float/int).
- Método __str__ formateado para salida amigable en consola.

```python
class Producto:
    def __init__(self, id_producto, nombre,
                 precio, pais_origen, existencias):
        self.id = id_producto
        self.nombre = nombre
        self.precio = float(precio)
        self.pais_origen = pais_origen
        self.existencias = int(existencias)

        # Enlaces de la lista doble
        self.anterior = None
        self.siguiente = None

    def __str__(self):
        return (f"ID: {self.id} | {self.nombre} | "
                f"¢{self.precio:,.2f} | "
                f"País: {self.pais_origen} | "
                f"Existencias: {self.existencias}")
```

2. Estructura de Cola FIFO Personalizada

Clase ColaProductos

Implementación manual sin utilizar collections.deque. Se usa un nodo simple para mantener operaciones O(1) en encolar.

- encolar(): Inserta al final.
- desencolar(): Remueve del frente.
- __iter__(): Permite iterar sin destruir el contenido.

```python
class ColaProductos:
    def __init__(self):
        self.frente = None
        self.final = None
        self.tamano = 0

    def encolar(self, producto):
        nuevo = NodoCola(producto)
        if self.frente is None:
            self.frente = nuevo
            self.final = nuevo
        else:
            self.final.siguiente = nuevo
            self.final = nuevo
        self.tamano += 1
```

3. Inserción y Eliminación en DLL

Gestión de Punteros

Validación de duplicados por ID y reconexión cuidadosa de punteros anterior y siguiente al eliminar.

```
# Reconectar enlaces al eliminar nodo
if nodo.anterior is not None:
    nodo.anterior.siguiente = nodo.siguiente
else:
    self.cabeza = nodo.siguiente # Era la cabeza

if nodo.siguiente is not None:
    nodo.siguiente.anterior = nodo.anterior
else:
    self.cola = nodo.anterior # Era la cola
```

Inserción al Final

Actualización bidireccional inmediata al insertar un nuevo producto.

```
def insertar(self, id_prod, nom, pre, pais, exi):
    if self.buscar(id_prod) is not None:
        return False # ID duplicado

    nuevo = Producto(id_prod, nom, pre, pais, exi)
    if self.cabeza is None:
        self.cabeza = nuevo
        self.cola = nuevo
    else:
        nuevo.anterior = self.cola
        self.cola.siguiente = nuevo
        self.cola = nuevo
    self.tamano += 1
```

4. Recorrido Recursivo de la Lista

Concepto Algorítmico

Demostración de recursividad aplicando un parámetro por defecto como centinela inicial.

- **Caso Inicial:** Inicia desde self.cabeza.
- **Caso Base:** Si nodo is None, finaliza la recursión.
- **Paso Recursivo:** Imprime el nodo e invoca mostrar_recursivo(nodo.siguiente).

```
def mostrar_recursivo(self, nodo="INICIO"):
    # Primera llamada: arrancamos en cabeza
    if nodo == "INICIO":
        if self.cabeza is None:
            print("\nLista vacía.")
            return
        self.mostrar_recursivo(self.cabeza)
        return

    # Caso base
    if nodo is None:
        return

    # Caso recursivo
    print(nodo)
    self.mostrar_recursivo(nodo.siguiente)
```

5. Cola de Reabastecimiento (FIFO)

Conversión de DLL a Cola

Filtra los productos cuya existencia ha llegado a 0 y los añade a la lista de compras prioritaria.

- Recorre la lista doble linealmente.
- Si existencias == 0, ejecuta encolar().
- Mantiene orden de detección FIFO.

```python
def generar_cola_reabastecimiento(self):
    """
    Genera cola con productos con existencias = 0.
    """
    cola_compras = ColaProductos()
    actual = self.cabeza

    while actual is not None:
        if actual.existencias == 0:
            cola_compras.encolar(actual)
        actual = actual.siguiente

    return cola_compras
```

6. Frecuencias y Reporte de Recuperación

Frecuencia por País

Cuenta importaciones y las ordena descendentemente mediante tuplas.

```python
def generar_frecuencia_paises(self):
    frecuencias = {}
    actual = self.cabeza
    while actual is not None:
        p = actual.pais_origen
        frecuencias[p] = frecuencias.get(p, 0) + 1
        actual = actual.siguiente
    return sorted(frecuencias.items(),
                  key=lambda x: x[1], reverse=True)
```

Reporte Financiero (.txt)

Calcula el valor total inventariado a recuperar hoy:

$$\text{Total}_{\text{Recuperar}} = \sum_{i=1}^{n} \text{Precio}_i \times \text{Existencias}_i$$

Genera un archivo .txt tabulado con totales formateados en colones (₡).

Resultados con Datos de Ejemplo

10

Productos Registrados

4

En Cola de Reabastecimiento

5

Países de Origen

¿Preguntas o Comentarios?

¡Muchas gracias por su atención!

Presentado por el Grupo de Trabajo

Luis Diego Fernández Quesada

Juan Alexander Aguilar Artola